

Prática orientada por conceitos e evidências sobre HPA (Horizontal Pod AutoScaling) utilizando Kubernetes local e kubernetes utilizando AWS

Aluno: Marcelo Eichi Miyaki

Data: 25/08/2026

Sistema Utilizado: Windows 11 com Docker Desktop e WSL 2 e Amazon AWS CloudShell

Mapa do roteiro

Seção	Conteúdo
1	P0 - Organizar a execução de trabalho intermediário e instalar kubernetes local
2	P1 - Iniciar minicube e habilitar metrics-server
3	P2 - Baixar a imagem php-apache e expor serviço
4	P3 - Horizontal Pod Autoscaling (HPA) local
5	P5 - Criando as Funções de Acesso (IAM Roles)
6	P6 - Criando o Control Plane do EKS
7	P7 - Adicionando os Nós de Trabalho (Node Groups)
8	P8 - Conectar e gerir o cluster via AWS CloudShell
9	P9 - Implantação e Exposição da Aplicação Teste
10	P10 - Baixar a imagem php-apache e expor como serviço no CloudShell
11	P11 - Horizontal Pod Autoscalling (HPA) no CloudShell
12	P12 - limpar o ambiente com segurança HPA CloudShell
13	Referências técnicas consultadas para esta escopo

Contexto de execução e cópia dos comandos

O local de execução é indicado no texto imediatamente antes de cada bloco:

- terminal Ubuntu: Ubuntu/WSL2;
- PowerShell: terminal do Windows;
- navegador: navegador da máquina do aluno;
- dentro do Pod: comando executado por kubectl exec.

Para evitar erros de cópia e colagem, os blocos de código contêm somente os comandos. Não há prefixos como HOST> ou PS> dentro dos blocos. Quando um bloco tiver várias linhas independentes, execute

uma linha por vez, na ordem apresentada. Blocos com estruturas Bash, como if, case ou for, devem ser copiados integralmente.

HPA Kubernetes local

Arquivos usados

Durante P0, os arquivos do pacote serão copiados para ~/psi5120/t2026. Depois disso, o roteiro usará esta estrutura:

SAÍDA / EXEMPLO

```
~/psi5120/t2026/
manifests/                                # arquivos limpos e executáveis
  00-php-apache.yaml
  01-autoscaling-declarative.yaml
manifests_comentados/                    # cópias para estudo, com comentários linha
a linha
  00-php-apache-comentado.yaml
  01-autoscaling-declarative-comentado.yaml
documentacao/
  PSI5120_Roteiro_Pratico_HPA.md
```

Os arquivos em manifests/ são aplicados ao cluster. As cópias em manifests_comentados/ contêm comentários YAML iniciados por #; esses comentários não alteram o objeto, mas tornam visível o papel de cada campo. Antes de cada kubectl apply, o roteiro solicita a leitura da versão comentada e, em seguida, a conferência do arquivo limpo que será aplicado.

P0 - organizar a execução de trabalho intermediário e instalar Kubernetes local

P0.1 - Identificar o ambiente e validar o Docker

Execute no terminal Ubuntu:

TERMINAL BASH

```
if grep -qi microsoft /proc/version 2>/dev/null || [ -n "${WSL_INTEROP:-}" ]; then
  echo "Ambiente detectado: Ubuntu/WSL2"
else
  echo "Ambiente detectado: Ubuntu nativo"
fi
```

Saída esperada em Windows com Ubuntu/WSL2:

SAÍDA / EXEMPLO

```
Ambiente detectado: Ubuntu/WSL2
```

ATENÇÃO

No Ubuntu/WSL2 com Docker Desktop, não execute `sudo systemctl enable --now docker`. A mensagem `Unit file docker.service does not exist` é compatível com esse arranjo: o Engine não está instalado como serviço dentro da distribuição.

No Windows:

15. abra o Docker Desktop;
16. aguarde até o Docker Engine estar em execução;
17. confirme o uso do backend WSL2;
18. abra Settings > Resources > WSL Integration;
19. habilite a distribuição Ubuntu usada na disciplina;
20. aplique as alterações e aguarde a reinicialização, quando solicitada.

No PowerShell:

POWERSHELL

```
wsl -l -v
```

SAÍDA / EXEMPLO

NAME	STATE	VERSION
* Ubuntu	Running	2

A coluna VERSION deve conter 2. O asterisco dessa saída indica a distribuição WSL padrão; ele não é o asterisco do Docker context.

No terminal Ubuntu/WSL2, verifique o cliente e o servidor:

TERMINAL BASH

```
docker version
```

SAÍDA / EXEMPLO

```
Client:
 Version:   29.x.x
 ...
Server: Docker Desktop
 Engine:
  Version:  29.x.x
  ...
```

O que docker version comprova:

- Client: o Docker CLI está disponível neste terminal;
- Server: o cliente alcançou o Engine;
- versões diferentes de Client e Server podem ser válidas, desde que sejam compatíveis.

Identifique o destino dos próximos comandos:

TERMINAL BASH

```
docker context show
docker context ls
```

SAÍDA / EXEMPLO

```
default
```

SAÍDA / EXEMPLO

NAME	DESCRIPTION	DOCKER_ENDPOINT
ERROR		
default *	Current DOCKER_HOST based configuration	unix:///var/run/docker.sock
desktop-linux	Docker Desktop ...	

Coluna	Significado
NAME	nome da configuração de conexão
*	contexto atualmente selecionado
DESCRIPTION	descrição registrada
DOCKER_ENDPOINT	endereço ou socket usado para alcançar o Engine
ERROR	erro detectado ao resolver o contexto, quando houver

Para inspecionar o contexto ativo, quando houver dúvida:

TERMINAL BASH

```
docker context inspect "$(docker context show)"
```

Localize Name, Endpoints e Host. Não é necessário interpretar todo o JSON nesta aula.

Consulte o Engine selecionado:

TERMINAL BASH

```
docker info
```

SAÍDA / EXEMPLO

```
Server Version: 29.x.x
Storage Driver: overlayfs
Operating System: Docker Desktop
OSType: linux
Architecture: x86_64
CPUs: 8
Total Memory: 7.5GiB
```

Os valores variam. Para esta prática, confirme principalmente:

- OSType: linux;
- CPU e memória compatíveis com a execução do Docker Desktop e do Minikube;
- ausência de erro de conexão com o daemon.

Um resumo copiável pode ser obtido com:

TERMINAL BASH

```
docker info | grep -E 'Server Version|Storage Driver|Operating System|OSType'
docker info | grep -E 'Architecture|CPUs|Total Memory'
```

Verifique o plugin Compose:

TERMINAL BASH

```
docker compose version
```

Execute o teste integrado:

TERMINAL BASH

```
docker run --rm hello-world
```

Na primeira execução, o Engine pode obter a imagem antes de criar o container. O texto final deve conter uma mensagem equivalente a:

SAÍDA / EXEMPLO

```
Hello from Docker!
This message shows that your installation appears to be working correctly.
```

Esse teste percorre o caminho: localizar imagem -> obter imagem, se necessário -> criar container -> executar processo -> mostrar saída -> remover o container por causa de --rm.

P0.2 - Encerrar recursos residuais

ATIVIDADE CONDICIONAL

Classificação: CONDICIONAL. Execute somente se houver recursos ativos

Pode ter sido deixado containers ou uma aplicação Compose em execução. Isso pode consumir memória, CPU e portas necessárias ao laboratório Kubernetes.

No terminal Ubuntu:

TERMINAL BASH

```
docker compose ps
```

Se não houver aplicação Compose ativa, a saída poderá ser vazia ou conter apenas cabeçalho:

SAÍDA / EXEMPLO

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
------	-------	---------	---------	---------	--------	-------

Se houver serviços ativos, aparecerão containers com STATUS semelhantes a running.

Se houver serviços ativos:

TERMINAL BASH

```
docker compose down
```

SAÍDA / EXEMPLO

```
[+] Running 2/2
✓ Container ... Removed
✓ Network ... Removed
```

Não use -v neste ponto, a menos que você queira apagar também volumes persistentes.

Confira os containers ainda ativos:

TERMINAL BASH

```
docker ps
```

Interpretação: Não é necessário apagar imagens Docker. O objetivo é evitar containers desnecessários em execução, consumo de recursos, portas ocupadas e confusão entre containers e o node criado pelo Minikube.

P0.3 - Verificar recursos mínimos e a porta 8080

Volte ao diretório pessoal:

TERMINAL BASH

```
cd ~
```

Verifique CPU, memória e disco:

TERMINAL BASH

```
nproc  
free -h  
df -h ~
```

SAÍDA / EXEMPLO

```
8  
  
              total        used        free      shared  buff/cache   available  
Mem:          15Gi         4.2Gi         6.1Gi        120Mi         5.0Gi         10Gi  
Filesystem    Size  Used Avail Use% Mounted on  
/dev/sdc      251G   42G   197G   18% /
```

O que observar:

- nproc: quantidade de CPUs lógicas.
- available em free -h: memória disponível para uso.
- Avail em df -h ~: espaço livre no filesystem do diretório pessoal.

Critério mínimo para esta prática:

- 2 CPUs disponíveis;
- 2 GiB de memória livre para o cluster, além do sistema e do Docker;
- espaço livre suficiente para imagens e containers.

Verifique se a porta 8080 está livre:

TERMINAL BASH

```
ss -ltn | grep ':8080' || echo "porta 8080 livre"
```

SAÍDA / EXEMPLO

```
porta 8080 livre
```

Se aparecer alguma linha com :8080, identifique o processo:

TERMINAL BASH

```
sudo ss -ltnp | grep ':8080'
```

No Windows, também pode haver uma aplicação Windows usando essa porta. Nesse caso, confira no PowerShell:

POWERSHELL

```
Get-NetTCPConnection -LocalPort 8080 -ErrorAction SilentlyContinue
```

Interpretação: A porta 8080 será usada pelo kubectl port-forward. Se estiver ocupada, o acesso local à aplicação poderá falhar mesmo que Kubernetes esteja funcionando.

P0.4 - Criar o diretório

No terminal Ubuntu:

TERMINAL BASH

```
mkdir -p ~/psi5120/t2026
cd ~/psi5120/t2026
mkdir -p manifests manifests_comentados documentacao pacote
pwd
find . -maxdepth 1 -type d | sort
```

SAÍDA / EXEMPLO

```
/home/seu_usuario/psi5120/t2026
.
./documentacao
./manifests
./manifests_comentados
./pacote
```

Interpretação: ~/psi5120/t2026 é o diretório local. Ele não é o cluster Kubernetes e não é um namespace Kubernetes.

Elemento	Função
~/psi5120/t2026	diretório local com arquivos da prática
perfil Minikube t-2026-i	cluster Kubernetes local
namespace php -apache	separação lógica dos objetos dentro do cluster
manifests/	arquivos YAML aplicados ao cluster
scripts/	scripts auxiliares da aula

P0.5 - Copiar os arquivos do pacote

Antes de extrair o pacote, instale unzip:

TERMINAL BASH

```
sudo apt update  
sudo apt install -y unzip
```

Quando o navegador é executado no Windows, o ZIP normalmente estará em C:\Users\<usuario>\Downloads. No Ubuntu/WSL2, essa pasta aparece sob /mnt/c/Users/....

Use o comando abaixo para detectar automaticamente o diretório do perfil Windows:

TERMINAL BASH

```
WINHOME=$(wslpath "$(cmd.exe /c echo %USERPROFILE% 2>/dev/null | tr -d '\r')")  
echo "$WINHOME"
```

SAÍDA / EXEMPLO

```
/mnt/c/Users/stkof
```

Defina o nome do pacote e confira se o arquivo existe:

TERMINAL BASH

```
PACOTE=psi5120_t2026_roteiro_e_software_2026_v20.zip  
ORIGEM_ZIP="$WINHOME/Downloads/$PACOTE"  
if test -f "$ORIGEM_ZIP"; then  
    echo "Pacote encontrado: $ORIGEM_ZIP"  
else  
    echo "Pacote não encontrado: $ORIGEM_ZIP"  
fi
```

Se o arquivo não existir basta mudar o nome do PACOTE=nome_baixado

Copie o ZIP para o filesystem Linux e extraia-o:

TERMINAL BASH

```
cp "$ORIGEM_ZIP" ~/psi5120/t2026/pacote/  
cd ~/psi5120/t2026/pacote  
unzip -oq "$PACOTE"
```

Copiar manifestos, versões comentadas e documentação para a área oficial de trabalho:

TERMINAL BASH

```
ORIGEM=$(find . -type d -name 04_software | head -1)  
DEST="$HOME/psi5120/t2026"  
echo "$ORIGEM"  
cp "$ORIGEM/manifests/*.yaml" "$DEST/manifests/"  
cp "$ORIGEM/manifests_comentados/*.yaml" "$DEST/manifests_comentados/"  
cp "$ORIGEM/documentacao/*.md" "$DEST/documentacao/"
```

SAÍDA / EXEMPLO

```
./psi5120_t2026_roteiro_e_software_2026_v20/04_software
```

Os arquivos limpos, comentados e o guia de leitura devem ser copiados para diretórios diferentes. Essa separação permite estudar os comentários sem perder uma versão enxuta para aplicação e comparação.

Não execute a prática diretamente em /mnt/c/Users/.../Downloads. O diretório oficial do trabalho intermediário é ~/psi5120/t2026.

P0.6 - Validar a estrutura

No terminal Ubuntu:

TERMINAL BASH

```
cd ~/psi5120/t2026
find manifests -maxdepth 1 -print | sort
find manifests_comentados -maxdepth 1 -print | sort
find documentacao scripts evidencias -maxdepth 2 -print | sort
```

SAÍDA / EXEMPLO

```
documentacao
documentacao/PSI5120_Roteiro_Pratico_HPA.md
manifests
manifests_comentados/00-php-apache.yaml
manifests_comentados/01-autoscaling-declarative.yaml
manifests_comentados
manifests_comentados/00-php-apache-comentado.yaml
manifests_comentados/01-autoscaling-declarative-comentado.yaml
```

P0.7 - Instalar utilitários, kubectl e Minikube

ATIVIDADE CONDICIONAL

Classificação: CONDICIONAL. Execute somente se as ferramentas ainda não estiverem instaladas.

Instale os utilitários necessários:

TERMINAL BASH

```
sudo apt update
sudo apt install -y curl ca-certificates
```

Identifique a arquitetura:

TERMINAL BASH

```
uname -m
```

SAÍDA / EXEMPLO

```
x86_64
```

ou:

SAÍDA / EXEMPLO

```
aarch64
```

Defina a arquitetura automaticamente:

TERMINAL BASH

```
ARCH=$(uname -m)
case "$ARCH" in
  x86_64) KARCH=amd64 ;;
  aarch64) KARCH=arm64 ;;
  *)
    echo "arquitetura não suportada neste roteiro: $ARCH"
    exit 1
  ;;
esac
echo "$KARCH"
```

SAÍDA / EXEMPLO

```
amd64
```

Instale kubectl:

TERMINAL BASH

```
KUBECTL_VERSION=$(curl -L -s https://dl.k8s.io/release/stable.txt)
echo "$KUBECTL_VERSION"
curl -LO "https://dl.k8s.io/release/${KUBECTL_VERSION}/bin/linux/${KARCH}/kubectl"
curl -LO
"https://dl.k8s.io/release/${KUBECTL_VERSION}/bin/linux/${KARCH}/kubectl.sha256"
echo "$(cat kubectl.sha256) kubectl" | sha256sum --check
chmod +x kubectl
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
kubectl version --client
```

SAÍDA / EXEMPLO

```
kubect1: OK  
Client Version: v1.xx.x
```

Instale Minikube:

TERMINAL BASH

```
curl -LO  
"https://storage.googleapis.com/minikube/releases/latest/minikube-linux-`${KARCH}`"  
sudo install minikube-linux-`${KARCH}` /usr/local/bin/minikube  
minikube version
```

SAÍDA / EXEMPLO

```
minikube version: v1.xx.x
```

P0.8 - Verificar as ferramentas instaladas

No terminal Ubuntu:

TERMINAL BASH

```
docker version --format '{{.Client.Version}} / {{.Server.Version}}'  
docker compose version  
kubect1 version --client  
minikube version
```

SAÍDA / EXEMPLO

```
29.2.1 / 29.2.1  
Docker Compose version v2.xx.x  
Client Version: v1.xx.x  
minikube version: v1.xx.x
```

Interpretação: Essa verificação confirma que existem as três camadas operacionais:

SAÍDA / EXEMPLO

```
Docker Engine -> fornece o ambiente de containers usado pelo driver  
Minikube -> cria e administra o cluster Kubernetes local  
kubect1 -> envia requisições à API do cluster selecionado
```

P0.9 - Verificar o contexto Kubernetes antes de criar o cluster

Conceitos em foco:

kubeconfig é a configuração usada pelo kubect1 para registrar clusters, identidades de acesso e

contextos. O arquivo padrão costuma ser ~/.kube/config, mas a variável KUBECONFIG ou a opção --kubeconfig podem alterar essa origem.

Kubernetes context é uma associação nomeada entre:

SAÍDA / EXEMPLO

```
cluster + usuário/credencial + namespace opcional
```

O contexto atual define para qual API Kubernetes o kubectl enviará o próximo comando. Ele não deve ser confundido com Docker context, perfil Minikube ou namespace.

No terminal Ubuntu:

TERMINAL BASH

```
kubectl config current-context 2>/dev/null || echo "nenhum contexto configurado"
kubectl config get-contexts 2>/dev/null || true
```

Saída esperada antes da criação do Minikube:

SAÍDA / EXEMPLO

```
nenhum contexto configurado
```

ou uma tabela semelhante a:

SAÍDA / EXEMPLO

CURRENT	NAME	CLUSTER	AUTHINFO	NAMESPACE
*	outro-local	outro-local	outro-user	producao

P1 - Iniciar minikube e habilitar metrics-server

P1.1 - criar o cluster minikube

Executar o seguinte comando no terminal WSL:

TERMINAL BASH

```
minikube start -p t-2026-i --driver=docker
```

Saída esperada antes da criação do Minikube:

SAÍDA / EXEMPLO

```
Using the docker driver based...
Starting "t-2026-i" primary control-plane...
Done! kubectl is now configured to use "t-2026-i" cluster and "default" namespace by default
```

O que observar:

- o contexto ativo deve ser t-2026-i;
- o node deve estar Ready;
- o cluster é local e de nó único, portanto não representa uma instalação de produção ou EKS.

Selecionar explicitamente o contexto:

TERMINAL BASH

```
kubectl config use-context t-2026-i
kubectl config current-context
```

Saída esperada:

SAÍDA / EXEMPLO

```
Switched to context "t-2026-i".
t-2026-i
```

P1.2 - Verificar o node criado pelo Minikube

No terminal Ubuntu:

TERMINAL BASH

```
kubectl get nodes -o wide
```

Saída esperada:

SAÍDA / EXEMPLO

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP
OS-IMAGE			KERNEL-VERSION			CONTAINER-RUNTIME
t-2026-i	Ready	control-plane	85s	v1.35.1	192.168.49.2	<none>
Debian GNU/Linux 12 (bookworm)			6.6.87.2-microsoft-standard-WSL2			docker://29.2.1

Os valores de AGE, VERSION, INTERNAL-IP, OS-IMAGE, KERNEL-VERSION e CONTAINER-RUNTIME podem variar.

P1.3 - Verificar os componentes do sistema

No terminal Ubuntu:

TERMINAL BASH

```
kubectl get pods -n kube-system
```

Saída esperada:

SAÍDA / EXEMPLO

NAME	READY	STATUS	RESTARTS	AGE
coredns-7d764666f9-mhnxg	1/1	Running	0	3m21s
etcd-t-2026-i	1/1	Running	0	3m28s
kube-apiserver-t-2026-i	1/1	Running	0	3m28s
kube-controller-manager-t-2026-i	1/1	Running	0	3m28s
kube-proxy-f79sn	1/1	Running	0	3m22s

kube-scheduler-t-2026-i	1/1	Running	0	3m28s
storage-provisioner	1/1	Running	1 (2m59s ago)	3m25s

Os componentes essenciais devem apresentar:

SAÍDA / EXEMPLO

READY = 1/1
STATUS = Running

P1.4 - habilitar metrics server

No terminal Ubuntu:

TERMINAL BASH

```
minikube -p t-2026-i addons enable metrics-server
```

Saída esperada:

SAÍDA / EXEMPLO

metrics-server is an addon maintained by Kubernetes. For any concerns contact minikube on GitHub.

You can view the list of minikube maintainers at:

<https://github.com/kubernetes/minikube/blob/master/OWNERS>

- Using image registry.k8s.io/metrics-server/metrics-server:v0.8.1

☀ The 'metrics-server' addon is enabled

Identificar se o addons foi habilitado

No terminal Ubuntu:

TERMINAL BASH

```
minikube -p t-2026-i addons list
```

Saída esperada:

SAÍDA / EXEMPLO

default-storageclass	t-2026-i	enabled	✓	Kubernetes
...				
metrics-server	t-2026-i	enabled	✓	Kubernetes
...				
storage-provisioner	t-2026-i	enabled	✓	minikube
...				

P2 - Baixar a imagem php-apache e expor como serviço

P2.1 - php-apache

Existem 2 possíveis meios:

- Usar comando direto
- utilizando o arquivo .yaml disponibilizado

Caminho a) Usar comando direto:

No terminal Ubuntu:

TERMINAL BASH

```
kubectl apply -f https://k8s.io/examples/application/php-apache.yaml
```

Saída esperada:

SAÍDA / EXEMPLO

```
deployment.apps/php-apache created
service/php-apache created
```

Nesse caso o arquivo .yaml hospedado no site oficial é baixado no cluster diretamente e não ocupa memória local

Caminho b) Utilizando o arquivo .yaml disponibilizado

Ir até o diretório Ubuntu onde está localizado a pasta /psi5120/t2026

No terminal Ubuntu:

TERMINAL BASH

```
kubectl apply -f manifests/00-php-apache.yaml
```

Saída esperada:

SAÍDA / EXEMPLO

```
deployment.apps/php-apache created
service/php-apache created
```

P2.2 - Confirmar Servidor php-apache ativo

No terminal Ubuntu:

TERMINAL BASH

```
kubectl get deployment php-apache -o yaml
```

Saída esperada:

SAÍDA / EXEMPLO

```
apiVersion: apps/v1
kind: Deployment
metadata:
...
  replicas: 1
  revisionHistoryLimit: 10
...
template:
  metadata:
    labels:
      run: php-apache
```



```
spec:
  containers:
  - image: registry.k8s.io/hpa-example
    imagePullPolicy: Always
    name: php-apache
    ports:
    - containerPort: 80
      protocol: TCP
    resources:
      limits:
        cpu: 500m
      requests:
        cpu: 200m
  ...
```

P3 - Horizontal Pod Autoscaling (HPA) local

P3.1 - Ativar HPA local

Existem 2 possíveis meios:

- Usar comando direto
- Utilizando o arquivo .yaml disponibilizado

Caminho a) Usar comando direto:

No terminal Ubuntu:

TERMINAL BASH

```
kubectl autoscale deployment php-apache --cpu=50% --min=1 --max=10
```

Saída esperada:

SAÍDA / EXEMPLO

```
horizontalpodautoscaler.autoscaling/php-apache created
```

Nesse caso o arquivo .yaml hospedado no site oficial é baixado no cluster diretamente e não ocupa memória local além disso é possível alterar:

- o estresse médio exigido de cpu para todos os pods (mudar 50% para outro valor), caso o estresse seja maior que o indicado será criado mais pods se o estresse for diminuindo então os pods diminuem
- quantidade mínima ao iniciar (mudar de 1 para outro valor)
- quantidade máxima de Pods (mudar de 10 para outro valor maior que o mínimo)

Caminho b) Utilizando o arquivo .yaml disponibilizado

Ir até o diretório Ubuntu onde está localizado a pasta /psi5120/t2026

No terminal Ubuntu:

TERMINAL BASH

```
kubectl apply -f manifests/autoscaling_declarative.yaml
```

Saída esperada:

SAÍDA / EXEMPLO

```
horizontalpodautoscaler.autoscaling/php-apache created
```

Nesse caso para realizar as alterações mencionadas anteriormente é necessário editar o arquivo `autoscaling_declarative.yaml` disponibilizado.

P3.2 - Confirmar HPA ativo e status atual do HPA

No terminal Ubuntu:

TERMINAL BASH

```
kubectl get hpa
```

Saída esperada:

SAÍDA / EXEMPLO

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
php-apache	Deployment/php-apache	cpu: 0%/50%	1	10	1	XXs

P3.3 - Monitorar o HPA e aumentar a carga

Ver o evento HPA ocorrer enquanto a requisição de dados aumenta é necessário usar o comando abaixo no terminal para monitorar e o Pod funcionar como um servidor e depois em outro terminal utilizar outro comando para que um Pod possa agir como cliente e ficar em loop infinito, resultando ao final 2 terminais abertos.

- **Terminal onde será monitorado:**

No terminal Ubuntu:

TERMINAL BASH

```
kubectl get hpa php-apache --watch
```

Saída esperada:

SAÍDA / EXEMPLO

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
php-apache	Deployment/php-apache	cpu: 0%/50%	1	10	1	XXs
...						

Assim que ativar o Pod como client e depois de passado alguns minutos ira aparecer novas linhas

- **Terminal onde Pod funcionará como cliente:**

Abrir um novo terminal e entrar na mesma versão de WSL usado anteriormente, assim que entrar ir na pasta PSI com o comando:

No terminal Ubuntu:

TERMINAL BASH

```
cd ~/psi5120/t2026
```

Iniciar o Pod para agir como cliente e ficar em loop infinito

No terminal Ubuntu:

TERMINAL BASH

```
kubectrl run -i --tty load-generator --rm --image=busybox:1.28 --restart=Never --  
/bin/sh -c "while sleep 0.01; do wget -q -O- http://php-apache; done"
```

Saída esperada:

SAÍDA / EXEMPLO

All commands and output from this session will be recorded in container logs,
including credentials and sensitive information passed through the command prompt.
If you don't see a command prompt, try pressing enter.
OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!
...

Assim que o monitoramento se estabilizar, recomenda-se parar esse terminal com Ctrl+C

P3.4 - Imagens referente ao trabalho HPA máquina local

- Escalamento do cluster

```
marcelo@Carinha-PC:~/psi5120/t2026$ kubectl apply -f autoscaling_declarative.yaml  
horizontalpodautoscaler.autoscaling/php-apache created
```

- Terminal onde será monitorado:

```
marcelo@Carinha-PC:~/psi5120/t2026$ kubectl get hpa  
NAME          REFERENCE          TARGETS          MINPODS  MAXPODS  REPLICAS  AGE  
php-apache    Deployment/php-apache  cpu: 0%/50%      1         10        1          91s  
marcelo@Carinha-PC:~/psi5120/t2026$ kubectl get hpa php-apache --watch  
NAME          REFERENCE          TARGETS          MINPODS  MAXPODS  REPLICAS  AGE  
php-apache    Deployment/php-apache  cpu: 0%/50%      1         10        1          110s  
php-apache    Deployment/php-apache  cpu: 69%/50%     1         10        1          2m30s  
php-apache    Deployment/php-apache  cpu: 69%/50%     1         10        2          2m45s  
php-apache    Deployment/php-apache  cpu: 161%/50%    1         10        2          3m30s  
php-apache    Deployment/php-apache  cpu: 161%/50%    1         10        4          3m45s  
php-apache    Deployment/php-apache  cpu: 88%/50%     1         10        4          4m30s  
php-apache    Deployment/php-apache  cpu: 85%/50%     1         10        4          5m30s  
php-apache    Deployment/php-apache  cpu: 85%/50%     1         10        7          5m45s  
php-apache    Deployment/php-apache  cpu: 53%/50%     1         10        7          6m30s  
php-apache    Deployment/php-apache  cpu: 51%/50%     1         10        7          7m30s  
php-apache    Deployment/php-apache  cpu: 47%/50%     1         10        7          8m30s  
php-apache    Deployment/php-apache  cpu: 48%/50%     1         10        7          9m30s  
php-apache    Deployment/php-apache  cpu: 51%/50%     1         10        7          10m  
php-apache    Deployment/php-apache  cpu: 48%/50%     1         10        7          11m  
php-apache    Deployment/php-apache  cpu: 51%/50%     1         10        7          12m  
php-apache    Deployment/php-apache  cpu: 46%/50%     1         10        7          13m  
php-apache    Deployment/php-apache  cpu: 50%/50%     1         10        7          14m  
php-apache    Deployment/php-apache  cpu: 51%/50%     1         10        7          16m  
php-apache    Deployment/php-apache  cpu: 14%/50%     1         10        7          17m  
php-apache    Deployment/php-apache  cpu: 0%/50%      1         10        7          18m  
php-apache    Deployment/php-apache  cpu: 0%/50%      1         10        7          22m  
php-apache    Deployment/php-apache  cpu: 0%/50%      1         10        2          22m  
php-apache    Deployment/php-apache  cpu: 0%/50%      1         10        2          23m  
php-apache    Deployment/php-apache  cpu: 0%/50%      1         10        1          23m  
marcelo@Carinha-PC:~/psi5120/t2026$ kubectl status -n t2026 -l
```

- Terminal onde Pod funcionará como cliente:

```
marcelo@Carinha-PC:~/psi5120/t2026$ kubectl run -i --tty load-generator --rm
--image=busybox:1.28 --restart=Never -- /bin/sh -c "while sleep 0.01; do wg
et -q -O- http://php-apache; done"
All commands and output from this session will be recorded in container logs
, including credentials and sensitive information passed through the command
prompt.
If you don't see a command prompt, try pressing enter.
OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!O
K!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK
!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!
OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!O
K!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK
!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!
OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!O
K!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK
K!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK
```

Pelas imagens acima é possível perceber que antes de iniciar com CPU em 0% havia apenas 1 Pod pois não havia nenhuma requisição. Durante o processo até os 4 min indicado na aba age quando a quantidade de pods na coluna REPLICAS não supria as requisições do cliente a CPU aumentou muito até chegar em 161,5%. Entre os 5 min e 7min na aba age já houve uma diminuição da CPU em todos os pods , praticamente metade da exigência de cpu em todos os pods, ate o momento que continuava aumentando a quantidade de Pods em REPLICAS até que após os 8 min o HPA já conseguiu estabilizar a CPU no parâmetro de 50% de exigência para todos os pods, o qual ficou ao final com 7 pods ativos ja sustentar a demanda do outro Pod agindo como cliente. Já estabilizado o monitoramento a partir dos 16 min na aba age foi feito uma parada na requisição do cliente. Percebeu-se que após esse tempo houve a diminuição da exigência da CPU em todos os pods até zerar o processamento e depois a quantidade de pods diminuiu até chegar em seu tamanho mínimo de um pod aos 23 min.

P4 - limpar o ambiente com segurança HPA local

P3.4 - Deletar HPA local, deletar cluster e servidor php-apache, parar o minicube e deletar

- Primeiro deletar o HPA local:

No terminal Ubuntu:

TERMINAL BASH

```
kubectl delete hpa php-apache
```

Saída esperada:

SAÍDA / EXEMPLO

```
horizontalpodautoscaler.autoscaling "php-apache" deleted from default namespace
```

- Deletar deployment e service de php-apache:

TERMINAL BASH

```
kubectl delete deployment php-apache
kubectl delete service php-apache
```

Saída esperada:

SAÍDA / EXEMPLO

```
deployment.apps "php-apache" deleted from default namespace
service "php-apache" deleted from default namespace
```

- Deletar cluster

No terminal Ubuntu:

TERMINAL BASH

```
kubectl config get-contexts
kubectl config delete-context t-2026-i
```

Saída esperada:

SAÍDA / EXEMPLO

```
CURRENT  NAME      CLUSTER  AUTHINFO  NAMESPACE
*        t-2026-i  t-2026-i  t-2026-i  default
warning: this removed your active context, use "kubectl config use-context" to select
a different one deleted context t-2026-i from /home/marcelo/.kube/config
```

- Parar e deletar minikube

No terminal Ubuntu:

TERMINAL BASH

```
minikube stop -p t-2026-i
minikube delete -p t-2026-i
minikube profile list
```

Saída esperada:

SAÍDA / EXEMPLO

```
👋 Stopping node "t-2026-i" ...
🔴 Powering off "t-2026-i" via SSH ...
🔴 1 node stopped.
🔥 Deleting "t-2026-i" in docker ...
🔥 Deleting container "t-2026-i" ...
🔥 Removing /home/marcelo/.minikube/machines/t-2026-i ...
💀 Removed all traces of the "t-2026-i" cluster.

🧑 Exiting due to MK_USAGE_NO_PROFILE: No minikube profile was found.
💡 Suggestion:
```

You can create one using 'minikube start'.

```
warning: this removed your active context, use "kubectl config use-context" to select
a different one deleted context t-2026-i from /home/marcelo/.kube/config
```

HPA Kubernetes usando AWS

Resumo Pré-requisitos & Arquitetura

Conta AWS Ativa com permissões para criar recursos de IAM, EKS e EC2.

Sem instalações locais: Todo o processo será feito no navegador via Console da AWS e cloudshell

Toda a execução foi feita usando a região us-east-1

Para selecionar essa região vá no canto superior direito, clique na região atual e selecione N.Virginia us-east-1.

P5 - Criando as funções de Acesso (IAM Roles)

O EKS necessita de permissões explícitas para gerir recursos na conta AWS. É necessário criar duas Roles distintas no IAM.

P5.1 - Criar a Role do Control Plane (eks_master_role)

Esta função permite que o plano de controle gerenciado pela AWS controle os recursos do cluster.

1. Aceda ao console da AWS e procure por IAM na barra de pesquisa superior.
2. No menu lateral esquerdo, clique em **Roles (Funções)** e selecione **Create role (Criar função)**.
3. Em Trusted entity type (Tipo de entidade confiável), selecione **AWS service**.
4. Em Use case (Caso de uso), selecione **EKS** no menu suspenso e marque a opção **EKS - Cluster**.

Clique em Next.

5. A política **AmazonEKSClusterPolicy** será associada automaticamente. Clique em Next.

6. Em Role name, insira **eks_master_role** e clique em **Create role**.

P5.2 - Criar a Role dos Worker Nodes (eks_workers_role)

Esta função permite que os nós (instâncias EC2) executem os contêineres e se comuniquem com a API do Kubernetes.

1. No menu do **IAM**, clique novamente em **Create role**.
2. Em Trusted entity type, selecione **AWS service**.
3. Em Use case, escolha **EC2** e clique em Next.
4. Na pesquisa de políticas, procure e selecione as **três políticas obrigatórias**:

- **AmazonEKSWorkerNodePolicy**
- **AmazonEC2ContainerRegistryReadOnly**
- **AmazonEKS_CNI_Policy**

5. Clique em Next.

6. Em Role name, insira **eks_workers_role** e finalize clicando em **Create role**.

P6 - Criando o Control Plane do EKS

Com as permissões criadas, é hora de instanciar o "cérebro" do cluster.

1. Na barra de pesquisa do console AWS, procure por **EKS (Elastic Kubernetes Service)**.

2. No menu esquerdo, clique em **Clusters** e depois no botão **Add cluster > Create**.

3. **Passo 1 – Custom configuration:**

- **Desabilite Use EKS Auto Mode**
- **Cluster configuration:**

Name: insira **intermediario-cluster-eks**.

Cluster IAM role: eks_master_role criada na Fase 1.1

- **Kubernetes version:** Selecione a versão recomendada (ex: 1.36 ou a mais recente).
- **Support type:** Standard support
- Na seção **Cluster access**, selecione:

Bootstrap cluster administrator access -> Allow cluster administrator access

Cluster authentication mode -> EKS API

- Secrets encryption manter desabilitada
- ARC Zonal shift manter desabilitado
- **Adicionar a tag Key** -> intermediario e **value** -> trabalho

Clique em Next.

4. **Passo 2 – Specify networking:**

- **VPC:** Selecione a VPC padrão da sua conta (ou uma VPC dedicada).
- **Subnets:** Certifique-se de ter pelo menos duas subnets selecionadas em Zonas de **Disponibilidade (AZs) diferentes** (ex: us-east-1a e us-east-1b).
- **Cluster endpoint access:** Selecione Public and private. Isto permite acesso à API via CloudShell e garante comunicação privada entre o Control Plane e os nós.

Clique em Next.

5. Passos 3 a 5 (Logging, Add-ons e Revisão): Avance mantendo as opções padrão até clicar em **Create**.

IMPORTANTE:

- Na etapa 4 em select add-ons confirmar em:
 - Community add-ons -> metrics server selecionado
- Na etapa 5 em confirmar se metrics server tem status ready to install

 **Tempo de espera:** A criação do Control Plane demora cerca de 10 a 15 minutos. Aguarde até que o estado do cluster mude de Creating para Active.

P7 - Adicionando os Nós de Trabalho (Node Groups)

Após o Control Plane estar Active, adicione as máquinas que irão executar os Pods/aplicações.

1. Na página do **cluster intermediario-cluster-eks**, vá na opção **Compute (Computação)**.

2. Na secção Node groups, clique em **Add node group** ou **Add**

3. **Passo 1 – Configure node group:**

- **Name:** Insira intermediario-worker-nodes.
- **Node IAM Role:** Selecione a eks_workers_role criada na Fase 1.2.
- **Adicionar a tag** Key -> intermediario e **value** -> trabalho

Clique em Next.

4. **Passo 2 – Set compute and scaling configuration:**

- **AMI type:** AMI type padrão EKS-optimized Linux exibido (Amazon Linux 2023 (x86_64) Standart (AL2023_x86_64_STANDART))
- **Capacity type:** On demand
- **Instance type:** Selecione **c7i-flex.large**

OBS: Instance type: t3.medium só funciona se a conta for paga, para ver outras instâncias free tier disponíveis vá na barra de pesquisa do console AWS em EC2 e no lado esquerdo vá em instâncias -> launch instâncias -> tipo de instância verá uma lista de free tiers que no momento verificado em 2026 possui 4 instâncias:

- t3.micro
- t3.small
- c7i-flex.large (mais próximo da necessidade desse escopo)
- m7i-flex.large

OBS2: No escopo feito poderia aumentar maximum size para 15 que iria funcionar o teste a CPU=50% ou escolher a instância m7i-flex.large

- **Disk size:** Mantenha em 20 GiB.
- **Node Group scaling configuration:**
 - **Desired size:** 1
 - **Minimum size:** 1
 - **Maximum size:** 10

Clique em Next.

5. Passo 3 – **Specify networking:** Mantenha as subnets selecionadas automaticamente e clique em Next.

6. Passo 4 – **Review and create:** Revise e clique em Create.

 **Tempo de espera: O provisionamento das instâncias EC2 demora cerca de 3 a 5 minutos.**

P8 - Conectar e gerir o cluster via AWS CloudShell

Com o cluster totalmente criado, utilize o **AWS CloudShell** integrado no navegador para interagir com o Kubernetes.

1. Abrir o AWS CloudShell:

- No topo do console da AWS (ao lado da barra de pesquisa), clique no ícone do terminal **>_ (AWS CloudShell)**.
- Aguarde alguns segundos até o ambiente Linux carregar.

2. Verificar ou Instalar o kubectl:

- Execute o comando para testar se o kubectl já está disponível:

TERMINAL CloudShell

```
kubectl version --client
```

Saída esperada:

SAÍDA / EXEMPLO

```
Client Version: v1.36.2-eks-8f14419  
Kustomize Version: v5.8.1
```

A versão acima pode mudar entretanto o importante é não dar o not found

Se o comando não for encontrado, instale-o com o seguinte bloco:

TERMINAL CloudShell

```
curl -O [https://s3.us-west-2.amazonaws.com/amazon-eks/1.30.0/2024-05-12/bin/linux/amd64/kubectl](https://s3.us-west-2.amazonaws.com/amazon-eks/1.30.0/2024-05-12/bin/linux/amd64/kubectl)

chmod +x ./kubectl
mkdir -p $HOME/bin && cp ./kubectl $HOME/bin/kubectl && export
PATH=$HOME/bin:$PATH
```

Saída esperada:

SAÍDA / EXEMPLO

```
Client Version: v1.36.2-eks-8f14419
Kustomize Version: v5.8.1
```

3. Configurar a Conexão ao Cluster (Kubeconfig):

- Execute o comando da AWS CLI para gerar as credenciais de acesso local no CloudShell:

TERMINAL CloudShell

```
aws eks update-kubeconfig --region us-east-1 --name intermediario-cluster-eks
```

Saída esperada:

SAÍDA / EXEMPLO

```
Added new context arn:aws:eks:us-east-1:744574783130:cluster/intermediario-cluster-eks
to /home/cloudshell-user/.kube/config
```

4. Validar a Comunicação:

- Liste os nós do cluster para confirmar que estão operacionais:

TERMINAL CloudShell

```
kubectl get nodes
```

Saída esperada:

SAÍDA / EXEMPLO

NAME	STATUS	ROLES	AGE	VERSION
ip-172-31-11-31.ec2.internal	Ready	<none>	19m	v1.36.2-eks-b3f9404

Resultado esperado: Deve surgir 1 nó listado com o estado Ready .

P9 - Implantação e Exposição da Aplicação de Teste

Para testar se o cluster está pronto para receber workloads e tráfego da Internet, vamos criar uma aplicação Nginx e expô-la através de um Load Balancer da AWS.

1. Criar o Deployment:

Este comando criará os Pods a correr o servidor Nginx.

TERMINAL CloudShell

```
kubectl create deployment nginx-teste --image=nginx
```

Saída esperada:

SAÍDA / EXEMPLO

```
deployment.apps/nginx-teste created
```

2. Expor a aplicação para a Internet:

Vamos criar um serviço do tipo LoadBalancer. O Kubernetes vai pedir à AWS para provisionar um balanceador de carga público apontado para os seus Pods.

TERMINAL CloudShell

```
kubectl expose deployment nginx-teste \
  --name=nginx-teste-service \
  --port=80 --target-port=80 \
  --type=LoadBalancer
```

Saída esperada:

SAÍDA / EXEMPLO

```
service/nginx-teste-service exposed
```

3. Obter o Endereço Público (DNS):

Verifique os detalhes do serviço recém-criado para descobrir o URL de acesso.

TERMINAL CloudShell

```
kubectl get service nginx-teste-service
```

Saída esperada:

SAÍDA / EXEMPLO

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP
nginx-teste-service	LoadBalancer	10.100.230.207	
a62df5a9db1db41a4a676ef3d280f49f-12040872.us-east-1.elb.amazonaws.com			80:31350/TCP
5m57s			

Nota: Na coluna EXTERNAL-IP, o estado pode aparecer como <pending> inicialmente. Aguarde cerca de 2 a 3 minutos e repita o comando até aparecer um endereço longo (ex: a1b2c3d4...us-east-1.elb.amazonaws.com).

4. Verificação no Browser:

Copie o endereço que apareceu no EXTERNAL-IP, abra um novo separador no browser do seu computador e cole o endereço. Ao premir **Enter**, deverá visualizar a página com a mensagem "**Welcome to nginx!**".

Nota: Abrir o endereço iniciando em http e não https

5. Limpeza do Teste:

Após validar o sucesso da configuração, apague a aplicação e o serviço para garantir que o Load Balancer da AWS é destruído e não gera custos desnecessários.

TERMINAL CloudShell

```
kubectl delete service nginx-teste-service
kubectl delete deployment nginx-teste
kubectl get service nginx-teste-service
```

Saída esperada:

SAÍDA / EXEMPLO

```
service "nginx-teste-service" deleted from default namespace
deployment.apps "nginx-teste" deleted from default namespace
Error from server (NotFound): services "nginx-teste-service" not found
```

Comandos utilizados no CloudShell

```
kubectl version --client
curl -O
https://s3.us-west-2.amazonaws.com/amazon-eks/1.30.0/2024-05-12/bin/linux/amd64/kubectl
chmod +x ./kubectl
mkdir -p $HOME/bin && cp ./kubectl $HOME/bin/kubectl && export PATH=$HOME/bin:$PATH
aws eks update-kubeconfig --region us-east-1 --name intermediario-cluster-eks
kubectl get nodes

kubectl create deployment nginx-teste --image=nginx
kubectl expose deployment nginx-teste --name=nginx-teste-service --port=80 --target-port=80
--type=LoadBalancer
kubectl get service nginx-teste-service
kubectl delete service nginx-teste-service
kubectl delete deployment nginx-teste
```

P10 - Baixar a imagem php-apache e expor como serviço no CloudShell

P10.1 - php-apache

Existem 2 possíveis meios:

- Usar comando direto
- utilizando o arquivo .yaml disponibilizado

Caminho a) Usar comando direto:

No terminal do CloudShell:

TERMINAL BASH

```
kubectl apply -f https://k8s.io/examples/application/php-apache.yaml
```

Saída esperada:

SAÍDA / EXEMPLO

```
deployment.apps/php-apache created
service/php-apache created
```

Nesse caso o arquivo .yaml hospedado no site oficial é baixado no cluster diretamente e não ocupa memória local

Caminho b) Utilizando o arquivo .yaml disponibilizado

Fazer upload do arquivo 00-php-apache.yaml

Ir até o diretório do CloudShell onde está localizado o arquivo 00-php-apache.yaml (geralmente se encontra na raiz)

Opcional: Pode ser criado uma pasta com o comando `mkdir` e depois pegar o arquivo upado com o comando `mv` nome do arquivo \local do diretório e partir disso entrar no diretório e executar os comandos abaixo, questão de organização no terminal do CloudShell

No terminal CloudShell:

TERMINAL BASH

```
kubectl apply -f 00-php-apache.yaml
```

Saída esperada:

SAÍDA / EXEMPLO

```
deployment.apps/php-apache created
service/php-apache created
```

P10.2 - Confirmar Servidor php-apache ativo

No terminal do CloudShell:

TERMINAL BASH

```
kubectl get deployment php-apache -o yaml
```

Saída esperada:

SAÍDA / EXEMPLO

```
apiVersion: apps/v1
kind: Deployment
metadata:
...
  replicas: 1
  revisionHistoryLimit: 10
...
template:
  metadata:
    labels:
      run: php-apache
  spec:
    containers:
      - image: registry.k8s.io/hpa-example
        imagePullPolicy: Always
        name: php-apache
        ports:
          - containerPort: 80
            protocol: TCP
        resources:
          limits:
            cpu: 500m
          requests:
            cpu: 200m
...

```

P11 - Horizontal Pod Autoscaling (HPA) no CloudShell

P11.1 - Ativar HPA no CloudShell

Existem 2 possíveis meios:

- Usar comando direto
- Utilizando o arquivo .yaml disponibilizado

Caminho a) Usar comando direto:

No terminal CloudShell:

TERMINAL BASH

```
kubectl autoscale deployment php-apache --cpu=50% --min1 --max=10
```

Saída esperada:

SAÍDA / EXEMPLO

```
Flag --cpu-percent has been deprecated, Use --cpu with percentage or resource quantity
format (e.g., '70%' for utilization or '500m' for milliCPU).
horizontalpodautoscaler.autoscaling/php-apache autoscaled
```

Nesse caso o arquivo .yaml hospedado no site oficial é baixado no cluster diretamente e não ocupa memória local além disso é possível alterar:

- o estresse médio exigido de cpu para todos os pods (mudar 50% para outro valor), caso o estresse seja maior que o indicado será criado mais pods se o estresse for diminuindo então os pods diminuem
- quantidade mínima ao iniciar (mudar de 1 para outro valor)
- quantidade máxima de Pods (mudar de 10 para outro valor maior que o mínimo)

Caminho b) Utilizando o arquivo .yaml disponibilizado

Fazer upload do arquivo 01-autoscalling-declarative.yaml

Ir até o diretório do CloudShell onde está localizado o arquivo 01-autoscalling-declarative.yaml (geralmente se encontra na raiz)

Opcional: Pode ser criado uma pasta com o comando mkdir e depois pegar o arquivo upado com o comando mv nome do arquivo \local do diretorio e partir disso entrar no diretorio e executar os comandos abaixo, questao de organização no terminal do CloudShell

No terminal CloudShell:

TERMINAL BASH

```
kubectl apply -f 01-autoscalling-declarative.yaml
```

Saída esperada:

SAÍDA / EXEMPLO

```
horizontalpodautoscaler.autoscaling/php-apache created
```

Nesse caso para realizar as alterações mencionadas anteriormente é necessário editar o arquivo 01-autoscalling-declarative.yaml disponibilizado.

P11.2 - Confirmar HPA ativo e status atual do HPA

No terminal CloudShell:

TERMINAL BASH

```
kubectl get hpa
```

Saída esperada:

SAÍDA / EXEMPLO

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
php-apache	Deployment/php-apache	cpu: 0%/50%	1	10	1	2m18s

P11.3 - Monitorar o HPA e aumentar a carga

Ver o evento HPA ocorrer enquanto a requisição de dados aumenta é necessário usar o comando abaixo no terminal para monitorar e o Pod funcionar como um servidor e depois em outro terminal utilizar outro comando para que um Pod possa agir como cliente e ficar em loop infinito, resultando ao final 2 terminais abertos.

- **Terminal onde será monitorado:**

No terminal CloudShell:

TERMINAL BASH

```
kubectl get hpa php-apache --watch
```

Saída esperada:

SAÍDA / EXEMPLO

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
php-apache	Deployment/php-apache	cpu: 0%/50%	1	10	1	5m19s
...						

Assim que ativar o Pod como client e depois de passado alguns minutos ira aparecer novas linhas

- **Terminal onde Pod funcionará como cliente:**

Abrir um novo terminal e entrar na mesma versão de WSL usado anteriormente, assim que entrar ir na na pasta onde encontrar os arquivos .yaml com o comando:

No terminal CloudShell:

TERMINAL BASH

```
cd~/nome do diretorio
```

Iniciar o Pod para agir como cliente e ficar em loop infinito

No terminal CloudShell:

TERMINAL BASH

```
kubectl run -i \
  --tty load-generator \
  --rm --image=busybox \
  --restart=Never \
  -- /bin/sh -c "while sleep 0.01; do wget -q -O- http://php-apache; done"
```

Saída esperada:

SAÍDA / EXEMPLO

```
All commands and output from this session will be recorded in container logs,
including credentials and sensitive information passed through the command prompt.
If you don't see a command prompt, try pressing enter.
OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!OK!
...
```


Assim que o monitoramento se estabilizar, recomenda-se parar esse terminal com Ctrl+C

P11.4 - Imagens referente ao trabalho HPA na Amazon AWS CloudShell

- **Escalamento do cluster**

```
psi5120-intermediario $ kubectl apply -f 01-autoscaling-declarative.yaml
horizontalpodautoscaler.autoscaling/php-apache created
```

- **Terminal onde será monitorado:**

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
php-apache	Deployment/php-apache	cpu: 0%/50%	1	10	1	2m18s
psi5120-intermediario	\$ kubectl get hpa	php-apache	--watch			
NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
php-apache	Deployment/php-apache	cpu: 0%/50%	1	10	1	5m19s
php-apache	Deployment/php-apache	cpu: 32%/50%	1	10	1	7m46s
php-apache	Deployment/php-apache	cpu: 250%/50%	1	10	1	8m1s
php-apache	Deployment/php-apache	cpu: 250%/50%	1	10	4	8m16s
php-apache	Deployment/php-apache	cpu: 106%/50%	1	10	5	8m31s
php-apache	Deployment/php-apache	cpu: 91%/50%	1	10	7	8m46s
php-apache	Deployment/php-apache	cpu: 68%/50%	1	10	10	9m1s
php-apache	Deployment/php-apache	cpu: 57%/50%	1	10	10	9m16s
php-apache	Deployment/php-apache	cpu: 61%/50%	1	10	10	9m31s
php-apache	Deployment/php-apache	cpu: 59%/50%	1	10	10	9m46s
php-apache	Deployment/php-apache	cpu: 64%/50%	1	10	10	10m
php-apache	Deployment/php-apache	cpu: 61%/50%	1	10	10	10m
php-apache	Deployment/php-apache	cpu: 59%/50%	1	10	10	10m
php-apache	Deployment/php-apache	cpu: 59%/50%	1	10	10	10m
php-apache	Deployment/php-apache	cpu: 61%/50%	1	10	10	11m
php-apache	Deployment/php-apache	cpu: 58%/50%	1	10	10	11m
php-apache	Deployment/php-apache	cpu: 62%/50%	1	10	10	11m
php-apache	Deployment/php-apache	cpu: 61%/50%	1	10	10	11m
php-apache	Deployment/php-apache	cpu: 60%/50%	1	10	10	12m
php-apache	Deployment/php-apache	cpu: 61%/50%	1	10	10	12m
php-apache	Deployment/php-apache	cpu: 61%/50%	1	10	10	12m
php-apache	Deployment/php-apache	cpu: 42%/50%	1	10	10	12m
php-apache	Deployment/php-apache	cpu: 2%/50%	1	10	10	13m
php-apache	Deployment/php-apache	cpu: 0%/50%	1	10	10	13m
php-apache	Deployment/php-apache	cpu: 0%/50%	1	10	10	17m
php-apache	Deployment/php-apache	cpu: 0%/50%	1	10	6	17m
php-apache	Deployment/php-apache	cpu: 0%/50%	1	10	1	18m

- **Terminal onde Pod funcionará como cliente:**

[illegible]

Pelas imagens acima é possível perceber que antes de iniciar com CPU em 0% havia apenas 1 Pod pois não havia nenhuma requisição. Durante o processo até os 8 min indicado na aba age quando a quantidade de pods na coluna REPLICAS não supria as requisições do cliente a CPU aumentou muito até chegar em 250%. Entre os 8m45s e 9m16s na aba age já houve uma diminuição da CPU em todos os pods , praticamente um quinto da exigência de cpu em todos os pods em relação aos 250%, até o momento que a quantidade de pods em REPLICAS não aumentava mais para atender as condições de 50% em todos os pods, o qual ficou ao final com 10 pods ativos atendendo a demanda do outro Pod agindo como cliente.

Já estabilizado o monitoramento a partir dos 12 min na aba age foi feito uma parada na requisição do cliente. Percebeu-se que após esse tempo houve a diminuição da exigência da CPU em todos os pods até zerar o processamento e depois a quantidade de pods diminuiu até chegar em seu tamanho mínimo de um pod aos 18 min.

Apesar da configuração do HPA estar setado em 50%, nessa configuração ele utilizou até o máximo permitido que era 10, entretanto se mudasse a **instance type** para **m7i-flex.large** ou outra mais robusta que **c7i-flex.large** ou aumentasse a quantidade máxima de Pods o o HPA iria manter a CPU de todos os Pods a 50% de estresse como ocorreu no experimento da máquina local.

P12 - limpar o ambiente com segurança HPA CloudShell

P12.1 - Deletar HPA CloudShell, servidor php-apache

- **Primeiro deletar o HPA CloudShell :**

No terminal CloudShell:

TERMINAL BASH

```
kubectl delete hpa php-apache
```

Saída esperada:

SAÍDA / EXEMPLO

```
horizontalpodautoscaler.autoscaling "php-apache" deleted from default namespace
```

- **Deletar deployment e service de php-apache:**

TERMINAL BASH

```
kubectl delete deployment php-apache  
kubectl delete service php-apache
```

Saída esperada:

SAÍDA / EXEMPLO

```
deployment.apps "php-apache" deleted from default namespace  
service "php-apache" deleted from default namespace
```

- **Remover o managed node group:**

AWS-CONSOLE> abra:

Amazon EKS -> Clusters -> intermediario-cluster-eks -> Compute

Selecione o node group e escolha a ação de exclusão.

Aguarde a remoção das instâncias EC2. Depois verifique:

- instâncias associadas ao grupo;
- volumes raiz;
- Auto Scaling Group e launch template gerados pelo EKS;
- interfaces de rede residuais.

O node group pode desaparecer da tela antes de todos os efeitos de cobrança aparecerem no Billing.

- **Remover o cluster:**

Depois que o Service externo e node group forem removidos:

AWS-CONSOLE> abra o cluster e escolha **Delete cluster**. Confirme o nome **intermediario-cluster-eks** e aguarde até ele desaparecer da lista.

- **Remover as roles IAM**

Depois da exclusão do cluster e do node group, remova as roles criadas exclusivamente para a prática:

eks_master_role

eks_workers_role

No IAM Console, abra cada **role**, **remova as políticas anexadas** e escolha **Delete**.

P13 - Referências técnicas consultadas para esse escopo

- Kubernetes Documentation: Horizontal Pod Autoscaler Walkthrough
<https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale-walkthrough/>
- AWS EKS Documentation: Scale pod deployments with Horizontal Pod Autoscaler
<https://docs.aws.amazon.com/eks/latest/userguide/horizontal-pod-autoscaler.html>
- Amazon EKS - Getting started with the AWS Management Console and AWS CLI:
<https://docs.aws.amazon.com/eks/latest/userguide/getting-started-console.html>
- Create an Amazon EKS cluster:
<https://docs.aws.amazon.com/eks/latest/userguide/create-cluster.html>

- Amazon EKS cluster IAM role:
<https://docs.aws.amazon.com/eks/latest/userguide/cluster-iam-role.html>
- Amazon EKS node IAM role:
<https://docs.aws.amazon.com/eks/latest/userguide/create-node-role.html>
- Connect kubectl to an EKS cluster:
<https://docs.aws.amazon.com/eks/latest/userguide/create-kubeconfig.html>
- Amazon EKS pricing:
<https://aws.amazon.com/eks/pricing/>
- Elastic Load Balancing pricing:
<https://aws.amazon.com/elasticloadbalancing/pricing/>